TIC TAC TOE IN PYTHON

Ultimate Beginner Friendly Walkthrough

Peacy Games

Part 1:
Grabbing tools from the toolbox

Before we can build, we need our tools.

```
import customtkinter as ctk
import random
```

1. **import customtkinter:** This is our toolkit for the window, buttons, and colors. To avoid typing this long word every time, we give it a nickname: ctk.
2. **import random:** This is the random number generator for our computer opponent so it can pick a random empty square later.

Part 2:
The Blueprint of the game

```
class TicTacToe(ctk.CTk):
    def __init__(self):
        super().__init__()
```

- **class TicTacToe:** We are creating a blueprint (a class) for our game.
- **(ctk.CTk):** We are telling the blueprint: "You are going to be a CustomTkinter window."
- **def __init__(self)::** This is the "engine start". Everything inside here runs immediately when the game starts.
- **self** means "myself". The program stores properties (like the game board) inside itself so it can access them later.
- **super().__init__():** This wakes up the invisible base window of CustomTkinter so we can start painting on it.

Part 3:
Setting up the window

Here we are decorating the house:

```
self.title("Tic Tac Toe - Epic Edition")
        self.geometry("400x580")
        self.resizable(False, False)

        ctk.set_appearance_mode("dark")
        ctk.set_default_color_theme("blue")
```

- We give the window a title and a fixed size (400 pixels wide, 580 pixels high).
- **resizable(False, False):** Forbids the user from dragging the window larger or smaller (otherwise our design would get messy).
- Then we turn on dark mode and set "blue" as the main color theme for our switches and highlights.

Part 4: Setting up the game's memory

The game needs to remember a few things:

```
self.current_player = "X"
        self.board = [["" for _ in range(3)] for _ in range(3)]
        self.buttons = [[None for _ in range(3)] for _ in range(3)]
        self.game_active = True

        self.setup_ui()
```

- current_player: Whose turn is it? We always start with "X".
- board: This is an invisible grid in the background. A list of 3 rows, each containing 3 empty text strings "". This is where the computer remembers where X and O are placed.
- buttons: This is where we will save the actual, clickable buttons.
- game_active = True: A switch that says: The game is currently running! If someone wins, we set this to False.
- Finally, we call self.setup_ui(). This is our signal: "Memory is ready, now build the visuals!"

Part 5:
Building the text and the AI Switch

We are now inside the function that builds the visuals:

```
def setup_ui(self):
        self.status_label = ctk.CTkLabel(
            self, text="Player X's turn", font=("Roboto", 28, "bold")
        )
        self.status_label.pack(pady=(20, 10))

        self.ai_switch = ctk.CTkSwitch(
            self, text="Computer Opponent (O)", font=("Roboto", 14, "bold")
        )
        self.ai_switch.pack(pady=(0, 10))
        self.ai_switch.select()
```

- **CTkLabel:** This is just plain text on the screen. We stick it to the top of the window using .pack(). The pady means "Padding Y" (spacing above and below).

- **CTkSwitch:** This is the On/Off toggle switch for the computer opponent. With .select(), we turn it "On" by default right at the start.

Part 6:
Preparing the grid frame

```
self.grid_frame = ctk.CTkFrame(self, fg_color="transparent")
self.grid_frame.pack(pady=10)
```

- A **CTkFrame** is like an empty picture frame. We place it in the middle of the window, make it see-through (transparent), and we will put our 9 buttons exactly inside this frame so they stay neatly together.

Part 7:
Spawning the 9 Buttons (The Loop)

This is the core of the visual interface! Instead of writing the exact same code 9 times for 9 buttons, we let the computer count:

```
for row in range(3):
            for col in range(3):
                btn = ctk.CTkButton(
                    self.grid_frame,
                    text="",
                    width=100, height=100, corner_radius=15,
                    font=("Roboto", 48, "bold"),
                    fg_color="#2b2b2b", hover_color="#3b3b3b",
                    command=lambda r=row, c=col: self.make_move(r, c)
                )
                btn.grid(row=row, column=col, padx=5, pady=5)
                self.buttons[row][col] = btn
```

- **for row in range(3):** Count rows 0, 1, and 2.
- **for col in range(3):** Count columns 0, 1, and 2.
- It builds a large, square button.
- The most important trick: command=lambda r=row, c=col: ... – Think of this like sticking a Post-it note on the button. We are telling it: "When you get clicked, remember your own row (r) and column (c) and call the function make_move."
- **.grid(...):** Places the button at its exact coordinates inside our picture frame.

Part 8:
The Restart Button

```
self.reset_btn = ctk.CTkButton(
            self, text="New Game", font=("Roboto", 18, "bold"),
            fg_color="#c93434", hover_color="#a32a2a",
            command=self.reset_game
        )
        self.reset_btn.pack(pady=20)
```

- At the very bottom, we stick a red button (#c93434 is the color code for a nice red). When it's clicked, it calls the self.reset_game function.

Part 9:
What happens on click?
(Anti-Cheat Protection)

Every time a button is clicked, we land here.

```
def make_move(self, row, col, is_ai=False):
        if self.ai_switch.get() == 1 and self.current_player == "O" and not is_ai:
            return
```

- **is_ai=False** means: Unless told otherwise, we assume a human clicked.
- The if statement checks: Is the switch turned on (get() == 1) AND is it O's turn AND was this move not made by the computer (not is_ai)?
- If yes: return (Abort!). This prevents you from clicking on behalf of the computer while it is "thinking".

Part 10:
Executing the move and writing it down

```
if self.board[row][col] == "" and self.game_active:
            self.board[row][col] = self.current_player

            text_color = "#3498db" if self.current_player == "X" else "#e74c3c"

            self.buttons[row][col].configure(
                text=self.current_player,
                text_color=text_color
            )
```

- We check: Is this specific spot in the invisible memory grid still empty ("") and is the game still active?
- If yes, we write the current player ("X" or "O") into our invisible memory (self.board).
- Then we pick a text color: Blue for X, Red for O.
- .configure: We update the actual button on the screen so it now shows the X or O in the correct color.

Part 11:
Do we have a winner?

```
if self.check_winner():
                self.status_label.configure(text=f"🏆 Player {self.current_player} wins!",
text_color="#2ecc71")
                self.game_active = False
            elif self.check_draw():
                self.status_label.configure(text="🤝 It's a draw!", text_color="#f1c40f")
                self.game_active = False
```

- We call the referee (self.check_winner()). If the referee says "Yes!", we change the top text to green (#2ecc71) and end the game (game_active = False).
- If nobody won, we check for a tie (self.check_draw()). If yes: Turn text yellow, end the game.

Part 12: Switching players and waking the computer

```
else:
                self.current_player = "O" if self.current_player == "X" else "X"

                if self.current_player == "X":
                    self.status_label.configure(text="Player X's turn", text_color="white")
                else:
                    if self.ai_switch.get() == 1:
                        self.status_label.configure(text="Computer (O) is thinking...",
text_color="gray")
                        self.after(600, self.ai_move)
                    else:
                        self.status_label.configure(text="Player O's turn", text_color="white")
```

- If the game continues, we swap shifts: Whoever was X becomes O (and vice versa).
- If it is now O's turn AND the switch is "On", we make the program wait for 600 milliseconds (self.after) and then call the computer's move function (ai_move). This makes it feel like the PC is actually thinking.

Part 13:
The Computer's Brain

```
def ai_move(self):
        empty_cells = []
        for r in range(3):
            for c in range(3):
                if self.board[r][c] == "":
                    empty_cells.append((r, c))

        if empty_cells and self.game_active:
            row, col = random.choice(empty_cells)
            self.make_move(row, col, is_ai=True)
```

- The computer grabs an empty notepad (empty_cells).
- It looks at every single square. If it's empty, it writes the coordinates down on the notepad.
- Are there any empty squares left? Then it blindly picks one at random using random.choice.
- It calls the click function, but whispers the secret password is_ai=True so the program knows: This isn't a cheater, this is the real computer!

Part 14:
The Referee
(Checking for a win)

Here, everything is simply double-checked:

```
def check_winner(self):
        for i in range(3):
            if self.board[i][0] == self.board[i][1] == self.board[i][2] != "":
                return True
            if self.board[0][i] == self.board[1][i] == self.board[2][i] != "":
                return True

        if self.board[0][0] == self.board[1][1] == self.board[2][2] != "":
            return True
        if self.board[0][2] == self.board[1][1] == self.board[2][0] != "":
            return True

        return False
```

- **The first loop checks all 3 horizontal rows and all 3 vertical columns. (!= "" means: "It cannot be empty; three empty squares do not win").**
- **Then it checks the two diagonal lines across the board.**
- **If it finds 3 identical symbols in a row, it yells True. If it searches everything and finds nothing, it yells False.**

Part 15:
Is it a draw?

```
def check_draw(self):
        for row in range(3):
            for col in range(3):
                if self.board[row][col] == "":
                    return False
        return True
```

- It looks at all 9 squares. As soon as it finds even a single empty square (""), it says False (No, it's not a draw yet, we can still play!). If it checks everything and finds zero empty squares, it says True (Yes, the board is entirely full).

Part 16:
The Red Reset Button

When you click "New Game", this function wipes the slate clean:

```
def reset_game(self):
        self.current_player = "X"
        self.game_active = True
        self.board = [["" for _ in range(3)] for _ in range(3)]
        self.status_label.configure(text="Player X's turn", text_color="white")

        for row in range(3):
            for col in range(3):
                self.buttons[row][col].configure(text="")
```

- **X goes first again.**
- **The game is active again.**
- **The invisible memory grid is completely erased.**
- **The text at the top is set back to white.**
- **All 9 visible buttons have their text removed (text="").**

Part 17:
The Ignition Key

This is placed at the very bottom of every Python user interface.

```
if __name__ == "__main__":
    app = TicTacToe()
    app.mainloop()
```

- **It means: "If this file is being run directly, then take the TicTacToe blueprint, build it, and start the infinite loop (mainloop)."**
- **The infinite loop keeps the window open until you click the X in the top corner to close it.**

Finally!

You can add
(next page)

🟢 Easy: Quick Wins

- A Score Tracker (Scoreboard)
  - What it is: Keep a running tally of how many times X has won, O has won, and how many draws there have been.
  - How you do it: Add three integer variables (e.g., self.score_x = 0). Create a new CTkLabel at the top. Every time someone wins in your make_move function, add +1 to their variable and update the label text.
- Winning Line Highlight
  - What it is: When a player wins, the three winning buttons change color (e.g., turn gold or bright green) so it's obvious how they won.
  - How you do it: Change your check_winner function. Instead of just returning True, make it return the specific row and column numbers of the three winning buttons. Then, use .configure(fg_color="green") on those specific buttons.
- Custom Player Names
  - What it is: Instead of "Player X", the game says "David's turn" or "Sarah wins!".
  - How you do it: Add a text input box (ctk.CTkEntry) above the board where players can type their names before the game starts. Save what they type into a variable and print that variable in your status label.

🟡 Medium: Stepping Up the Logic

- The "Undo" Button
  - What it is: A button that lets you take back your last move if you made a mistake.
  - How you do it: Create a list called self.history = []. Every time someone clicks a button, save the (row, col) to that list. If "Undo" is clicked, look at the last item in the list, change that button back to "", and switch the current player back.
- Sound Effects
  - What it is: A satisfying "pop" sound when you place an X or O, and a victory fanfare when someone wins.
  - How you do it: You will need to install a new library like pygame (specifically pygame.mixer). You load a short .wav or .mp3 file and trigger sound.play() inside your make_move function.
- Difficulty Switch for the AI
  - What it is: A segmented button where the user can choose if the computer is "Dumb" or "Smart".
  - How you do it: If "Dumb" is selected, the computer uses your current random move logic. If "Smart" is selected, you add logic where the computer first checks if it can win on the next move. If it can, it takes that spot instead of a random one.

🔴 Hard: Boss Level

- An Unbeatable AI (The Minimax Algorithm)
  - What it is: The computer looks ahead at every single possible future move until the end of the game. It will never lose—the best you can do against it is a draw.
  - How you do it: You have to write a "recursive" function (a function that calls itself) called Minimax. It scores game states: +10 if the AI wins, -10 if the human wins.
- Custom Board Sizes (4x4 or 5x5)
  - What it is: Let the user play on a massive grid where they need to get 4 or 5 in a row to win.
  - How you do it: You have to replace all the hardcoded 3s in your loops with a variable like self.board_size. You also have to completely rewrite your check_winner function so it can dynamically check lines of any length.

Complete Code

```
import customtkinter as ctk
import random

class TicTacToe(ctk.CTk):
    def __init__(self):
        super().__init__()

        self.title("Tic Tac Toe - Epic Edition")
        self.geometry("400x580") # Slightly taller to fit the switch
        self.resizable(False, False)

        ctk.set_appearance_mode("dark")
        ctk.set_default_color_theme("blue")

        self.current_player = "X"
        self.board = [["" for _ in range(3)] for _ in range(3)]
        self.buttons = [[None for _ in range(3)] for _ in range(3)]
        self.game_active = True

        self.setup_ui()
```

```
def setup_ui(self):
        self.status_label = ctk.CTkLabel(
            self,
            text="Player X's turn",
            font=("Roboto", 28, "bold")
        )
        self.status_label.pack(pady=(20, 10))

        # NEW: The AI switch
        self.ai_switch = ctk.CTkSwitch(
            self,
            text="Computer Opponent (O)",
            font=("Roboto", 14, "bold")
        )
        self.ai_switch.pack(pady=(0, 10))
        self.ai_switch.select() # AI is enabled by default

        self.grid_frame = ctk.CTkFrame(self, fg_color="transparent")
        self.grid_frame.pack(pady=10)

        for row in range(3):
            for col in range(3):
                btn = ctk.CTkButton(
                    self.grid_frame,
                    text="",
                    width=100,
                    height=100,
                    corner_radius=15,
                    font=("Roboto", 48, "bold"),
                    fg_color="#2b2b2b",
                    hover_color="#3b3b3b",
                    command=lambda r=row, c=col: self.make_move(r, c)
                )
                btn.grid(row=row, column=col, padx=5, pady=5)
                self.buttons[row][col] = btn

        self.reset_btn = ctk.CTkButton(
            self,
            text="New Game",
            font=("Roboto", 18, "bold"),
            fg_color="#c93434",
            hover_color="#a32a2a",
            command=self.reset_game
        )
        self.reset_btn.pack(pady=20)
```

```
def make_move(self, row, col, is_ai=False):
        # Block clicks if it is the AI's turn and a human tries to click
        if self.ai_switch.get() == 1 and self.current_player == "O" and not is_ai:
            return

        if self.board[row][col] == "" and self.game_active:
            self.board[row][col] = self.current_player

            text_color = "#3498db" if self.current_player == "X" else "#e74c3c"

            self.buttons[row][col].configure(
                text=self.current_player,
                text_color=text_color
            )

            if self.check_winner():
                self.status_label.configure(
                    text=f"🏆 Player {self.current_player} wins!",
                    text_color="#2ecc71"
                )
                self.game_active = False
            elif self.check_draw():
                self.status_label.configure(
                    text="🤝 It's a draw!",
                    text_color="#f1c40f"
                )
                self.game_active = False
            else:
                # Switch players
                self.current_player = "O" if self.current_player == "X" else "X"

                # Update status text and trigger AI move
                if self.current_player == "X":
                    self.status_label.configure(text="Player X's turn", text_color="white")
                else:
                    if self.ai_switch.get() == 1:
                        self.status_label.configure(text="Computer (O) is thinking...",
text_color="gray")
                        # Delays the AI by 600ms to make it feel realistic
                        self.after(600, self.ai_move)
                    else:
                        self.status_label.configure(text="Player O's turn", text_color="white")
```

```
def ai_move(self):
        empty_cells = []
        for r in range(3):
            for c in range(3):
                if self.board[r][c] == "":
                    empty_cells.append((r, c))

        if empty_cells and self.game_active:
            row, col = random.choice(empty_cells)
            self.make_move(row, col, is_ai=True)

    def check_winner(self):
        for i in range(3):
            if self.board[i][0] == self.board[i][1] == self.board[i][2] != "":
                return True
            if self.board[0][i] == self.board[1][i] == self.board[2][i] != "":
                return True

        if self.board[0][0] == self.board[1][1] == self.board[2][2] != "":
            return True
        if self.board[0][2] == self.board[1][1] == self.board[2][0] != "":
            return True

        return False

    def check_draw(self):
        for row in range(3):
            for col in range(3):
                if self.board[row][col] == "":
                    return False
        return True

    def reset_game(self):
        self.current_player = "X"
        self.game_active = True
        self.board = [["" for _ in range(3)] for _ in range(3)]

        self.status_label.configure(text="Player X's turn", text_color="white")

        for row in range(3):
            for col in range(3):
                self.buttons[row][col].configure(text="")

if __name__ == "__main__":
    app = TicTacToe()
    app.mainloop()
```

by

Peacy Games